

Lab 4 — Sprint 2: Content Creation with Factories

Where you are

In your Instructors video, we'll put Sprint 2 on your board. It's 3 stories, ten points, one goal.

A dungeon level is a **place, not a number.**

Install the starter

Unpack `lab04-factory-starter.zip` into your `dungeonforge` clone. It adds:

- an items package — `Item`, `Weapon`, `Armor`, `Potion`, `Treasure`, `Chest` — so a theme has loot to hand back. These are deliberately minimal; Week 9's Decorator Pattern will decorate these, Week 13 will make them Composites. **Don't over-build them now.**
- `src/main/resources/data/monsters.json` — Contains 15 blueprints across 3 themes. This is CONTENT, not code. You shouldn't spend this week inventing `dungeonforge` lore.
- `GameConfig.readResource(...)`, is now public, so your factory can load that file the same way `config` is loaded.

Part 0 — Feel the problem • 5 pts

Before any code, on `main`:

```
mvn -q exec:java > before.txt
grep -rn "new Monster(" src/main/java
```

Then read `before.txt` and answer, in `docs/evidence.md`:

1: Which species appear in level 1, and which on level 3?

HINT: They're the same 4. **Descending levels changes the numbers and nothing else.**

Open `Room.java` and look at **FLAVORS**:

That's crypt prose, compiled into the `Room` class. A forge level would describe itself as a damp tomb. Now you see the problem, so let's find the potion to cure it!

Branch: ``git switch -c week-04-factory``

Part A — US-2.1: Monsters come from data · 20 pts

3 points. Build `MonsterDef` (plain data) and `MonsterFactory` (the one place `new Monster` lives), loading blueprints from `monsters.json`.

```
public Monster create(String id, int depth) {
    MonsterDef d = blueprints.get(id);
    if (d == null) { /* AC4: unknown ids must not crash */ }
    // scale by depth, add seeded variance, return new Monster(...)
}
```

Use a **registry** `Map`, not an if/else chain over ids. Then delete `spawn()`.

The proof: add a monster to `monsters.json` — **YOU invent one** — and watch it appear in the game **without touching a single** `.java` file. Record that in your evidence.

Call it what it is. This is a ****Simple Factory****, an idiom, *****not***** a Gang of Four design pattern. Part D3 will ask you to explain what it ***can't do***.

Part B — US-2.2: Levels have character · 25 pts

4 points. The heart of the week.

`ThemeKit` is an **Abstract Factory** — an interface producing a *matched family*:

```
public interface ThemeKit {
    String themeName();
    Monster createMonster(int depth);
    Monster createBoss(int depth);
    Item    createLoot(int depth);
    String  createRoomFlavor();
}
```

Three implementations of `ThemeKit`: `CryptThemeKit`, `ForgeThemeKit`, `FrostThemeKit`. Plus a `ThemeRegistry` — the one class that knows which themes exist and maps depth to kit.

The constraint that makes this Abstract Factory rather than Factory Method: a

`CryptThemeKit` can *never* hand back a forge monster or forge loot. The kit guarantees the set goes together. Your unit tests will assert this over many draws.

Then move `FLAVORS` out of `Room` — a room shouldn't know what it sounds like — and give `DungeonLevel` a theme name.

Bosses need filtering. *The blueprint data has a `boss` flag. Ordinary encounters draw from the non-boss pool; a kit asks for its boss by name. Otherwise a Bone Tyrant turns up in a level-one corridor, which I promise will happen to you before it doesn't.*

Part C — US-2.3: Rooms differ · 20 pts

3 points. RoomPopulator is a **Factory Method**:

```
public final void populate(Room room, int depth) {    // FIXED skeleton
    room.setFlavor(theme.createRoomFlavor());
    for (Monster m : createEncounter(depth)) room.addMonster(m);
    Chest chest = createChest(depth);
    if (chest != null && !chest.isEmpty()) room.setChest(chest);
}

protected abstract List<Monster> createEncounter(int depth);    // DEFERRED
protected Chest createChest(int depth) { return null; }          // HOOK
```

Three Room subclasses that Implement the Factory Method:

1. Standard (0 to N monsters, no chest)
2. Treasure (one guard, a chest - it overrides the hook)
3. Boss (a boss, an escort, a hoard).

`populate()` is `final` on purpose, WHY? The *order* is the invariant this class protects, and a subclass that could override it could break the order. Your tests will assert the modifier by reflection.

Notice the two patterns composing. *The populator decides ****how many****; the kit decides ****which kind****. Two different axes of variation, two different patterns, one room. That's the sentence to put in your PR description. 😊*

Part D — The "Which Factory?" Clinic · 25 pts 🔍

docs/factory-clinic.md.

D1 — The experiment (8 pts). Add a **fourth theme**. Then count:

1. how many new files it required
2. how many modified
3. did GameWorld need to change?

Paste ``git diff -stat``.

Then set `dungeonDepth: 4`

1. run it
2. see your theme
3. then set it back before your pull-request.

*The Abstract Factory's claim is **"adding a family is cheap."** Claims like that get measured, not believed.*

D2 — Classification (12 pts). Six scenarios. Which of the three applies — or **none**?

1. Two rows are traps: one is a different pattern entirely, one isn't a pattern at all.

D3 — The distinction (5 pts). What can Factory Method do that Simple Factory cannot?

1. Don't define them. **Describe a change someone might ask for**, and explain why it's easy with one and awkward with the other.

D4 — One honest question. What's still blurry?

Part E — Close the sprint · 5 pts

DoD on every story · evidence complete · PR with CI green · Closes #N · tag `v0.4.0` · calibration table · docs/retro/sprint-02.md complete

`FactoryTest` includes a regression test for Week 3's determinism. *Same seed, same world.*
Refactoring without breaking last week's guarantee is the actual professional skill being practiced here.

Deliverables checklist

- ☐ docs/evidence.md — before-state, greps, after-state for all three stories
- ☐ factory/MonsterDef, MonsterFactory — loads `monsters.json`, one new `Monster`
- ☐ factory/ThemeKit + three kits + ThemeRegistry
- ☐ factory/RoomPopulator + three subclasses, `populate()` final
- ☐ `GameWorld.spawn()` **deleted**; `Room.FLAVORS` **deleted**
- ☐ `DungeonLevel` knows its theme; Main prints it
- ☐ `FactoryTest` passing, CI green
- ☐ docs/factory-clinic.md — D1 measured, D2 classified, D3 explained
- ☐ PR merged, tagged `v0.4.0`, calibration + retro filled in

Grading (100 pts)

Component	Pts
Part 0 — problem demonstrated first	5
Part A — Simple Factory, data-driven	20
Part B — Abstract Factory, matched families	25
Part C — Factory Method, final skeleton + hook	20
Part D — the clinic	25
Part E — sprint closed properly	5